





Ernesto Igor Laura Mamani
Ingeniero de Sistemas (Cs de la
Computación)
AI Engineer

Hobbies:

- Natación
- Videojuegos

Cargo Actual y Experiencia Docente

 **Interseguro** Generative AI Lead

Lidero un equipo de implementación de soluciones
basadas en IA Generativa.



Ministerio
de Salud



an NTT DATA Company

NTT DATA



Estudios

- Ingeniería de Sistemas

- Máster en Data Science
- Diplomado en Business Intelligence & Big data



<https://www.linkedin.com/in/ernesto-laura-mamani-9312714b/>

Conociendo tu Background

1. Nombre completo y país.
2. Tu formación (Ej: Ing en..., Lic en ...).
3. Dónde laboras y que actividades realizas.
4. Expectativas del programa.
5. Experiencia con IA y/o Python.

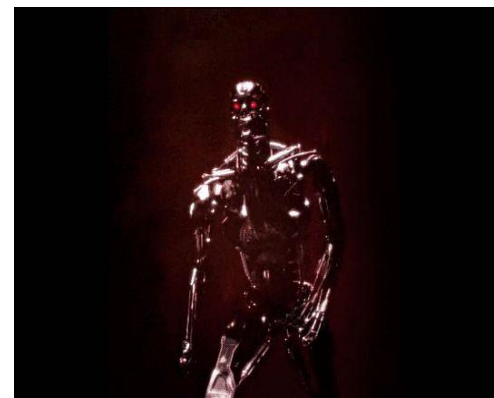
- Mantener el micrófono apagado en caso no vayan a consultar.
- Preguntar en caso que tengan dudas.
- Mantenerse atento a la clase. (Hacer el mayor esfuerzo)

Módulo	Sesiones
1. Fundamentos	1
2. Arquitectura de soluciones LLM's y Fundamentos de LLMOps	2
3. Langchain LLM's Processing, Data Management & RAG (Retrieval Augmented Generation) System	5
4. LLamaIndex y LlamaCloud - RAG (Retrieval Augmented Generation)	2
5. LLM's Crew AI Agentic Framework	3
6. Agentic Low Code with n8n (RAG)	1
7. LLM's Monitoring & Security con LangFuse	2
8. Taller de Agente IA de Voz in Real Time	1
9. Proyecto Integrador (end to end)	1

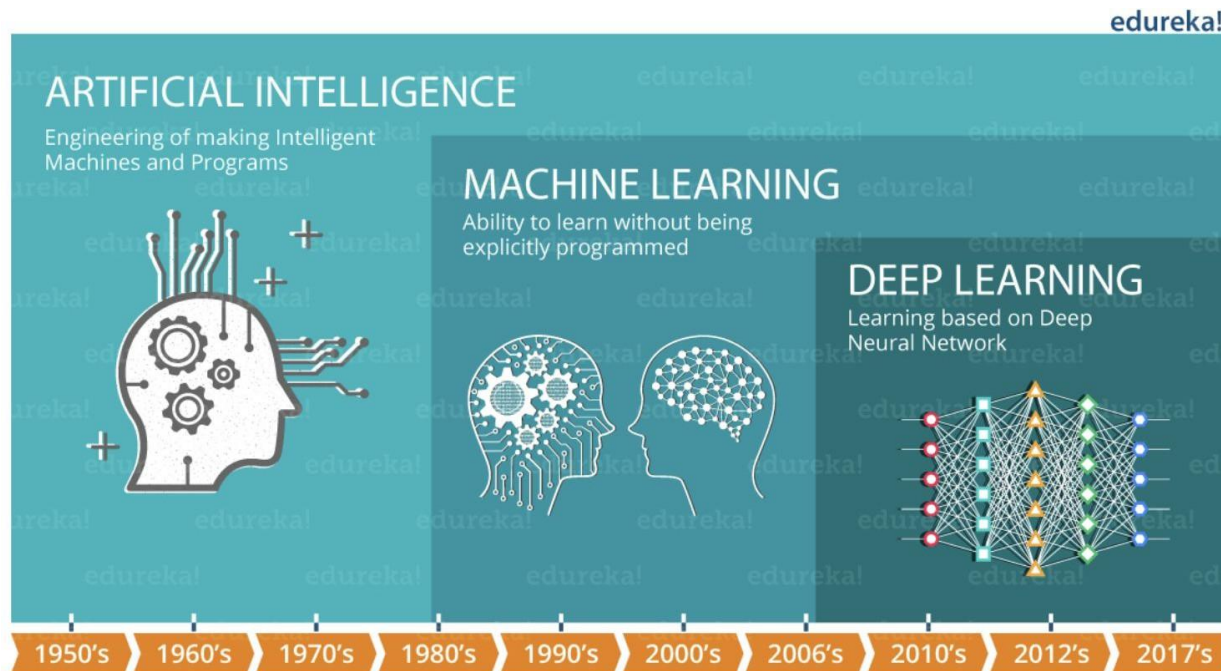
¿Qué es Inteligencia Artificial

¿Qué es la Inteligencia Artificial?

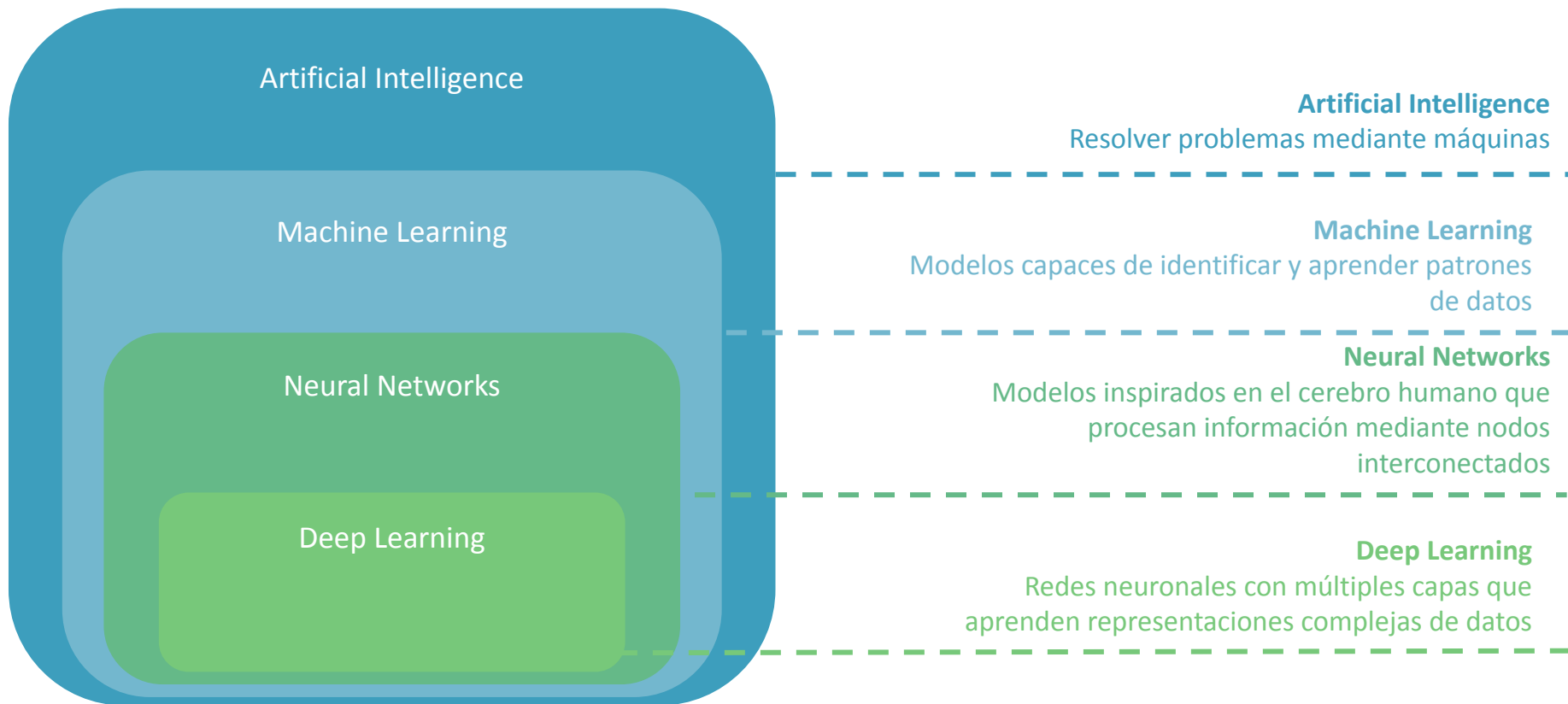
<<La habilidad de los ordenadores para hacer actividades que normalmente requieren inteligencia humana>> (Rouhiainen, 2018)



Fuente: Google imágenes



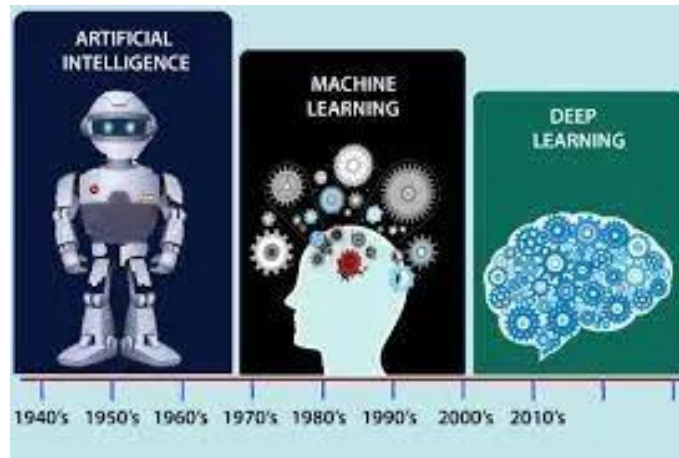
Fuente: <https://www.edureka.co/blog/what-is-deep-learning>



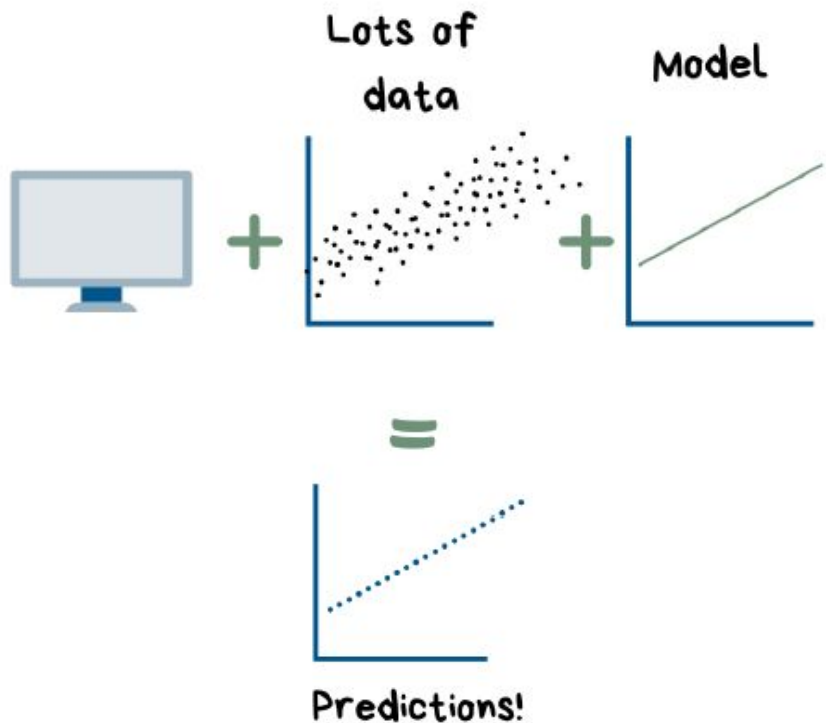
¿Qué es Machine Learning?

“Machine learning is a process by which a machine can learn on its own, without being explicitly programmed”. Machine Learning is a form of artificial intelligence that teaches computers to learn and make decisions from data. (1)

Machine learning is an application of artificial intelligence (AI) that provides systems the ability to automatically learn and improve from experience without being explicitly programmed (2)



Fuente: (2)



Entrenamiento

Predicción

Language Model

Un modelo de lenguaje codifica información estadística sobre uno o más idiomas. Intuitivamente, esta información nos indica la probabilidad de que una palabra aparezca en un contexto determinado.

Ejemplo: dado el contexto «Mi color favorito es __», un modelo de lenguaje que codifica el español debería predecir «azul» con más frecuencia que «carro».

La naturaleza estadística de los idiomas se descubrió hace siglos:

- 1905: “The Adventure of the Dancing Men” - *Sherlock Holmes*
- 1951: “Prediction and Entropy of Printed English” - *Claude Shannon*

Unidad básica de un modelo de lenguaje (caracter, palabra o parte de una palabra). Ejemplos:

GPT-4o & GPT-4o-mini:

No puedo esperar a construir aplicaciones con IA.

GPT-3.5 & GPT-4:

- 1 No puedo esperar a construir aplicaciones con IA.
- 2 las aplicaciones
- 3 las aplicaciones son buenas
- 4 aplicaciones

} Tokenización

Vocabulario: conjunto de todos los tokens de un modelo.

Ejemplo:

- Mixtral 7B: 32000
- GPT-4: 100256



token



caracter



palabra

História de los Large Language Models (LLMs)

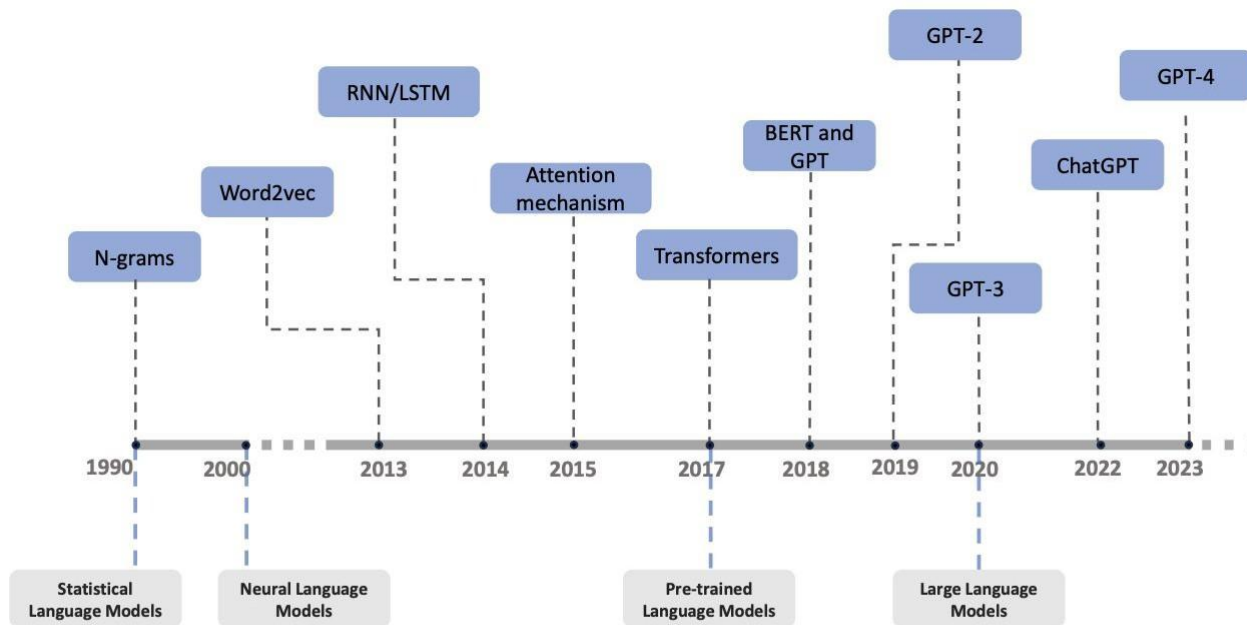
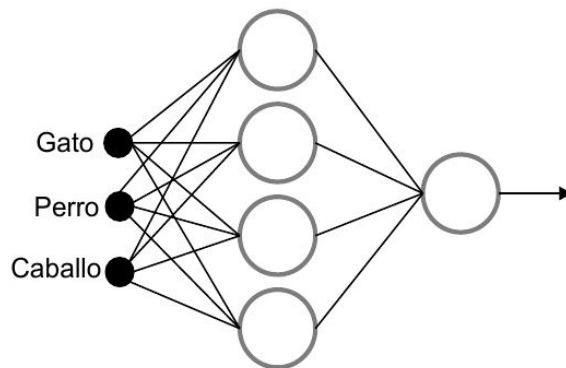


Figure 1: History and development of language models.

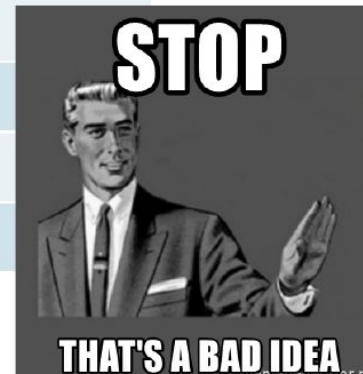
¿Cómo codificarías esta variable de entrada a una red neuronal?

Variable1	Gato	Perro	Caballo
Gato	1	0	0
Perro	0	1	0
Gato	1	0	0
Caballo	0	0	1

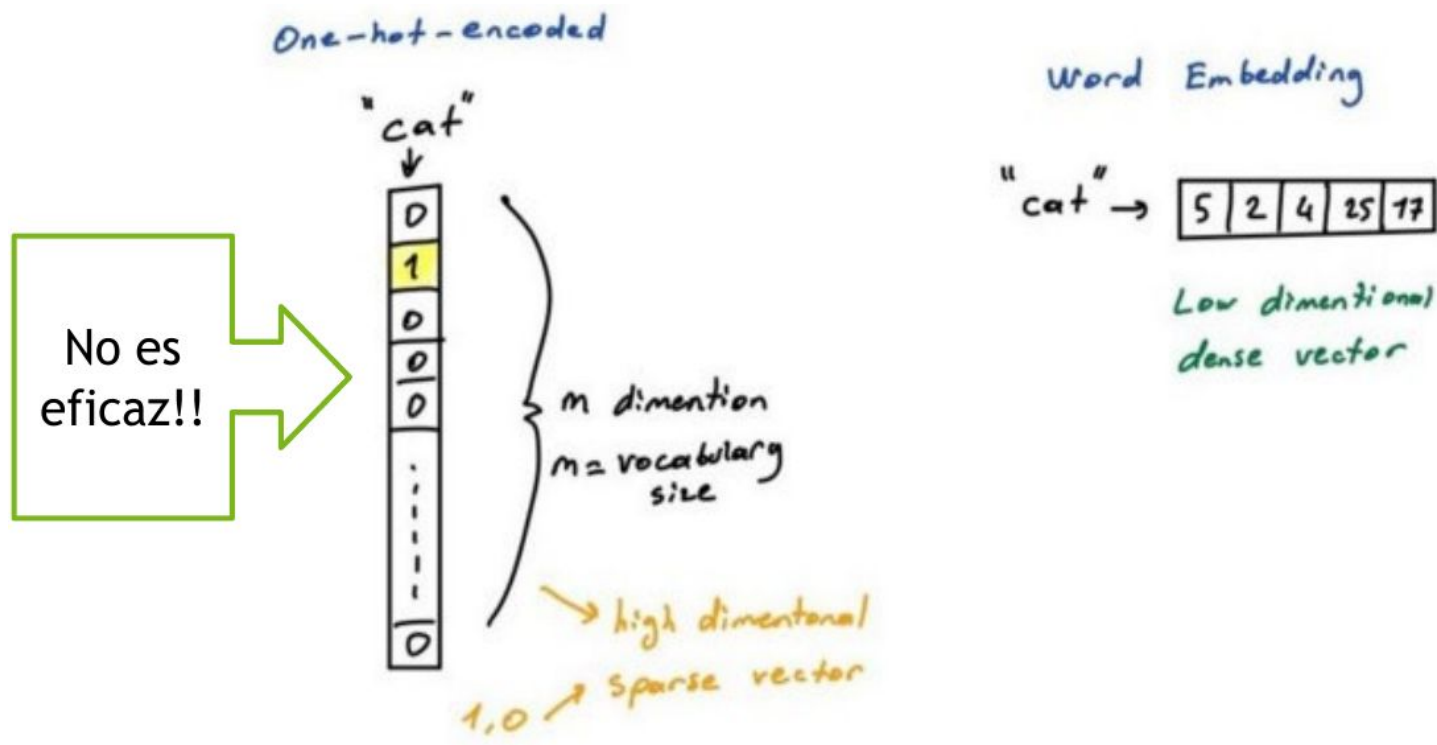


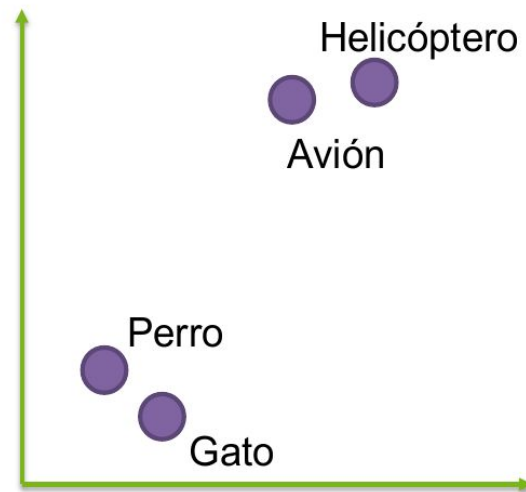
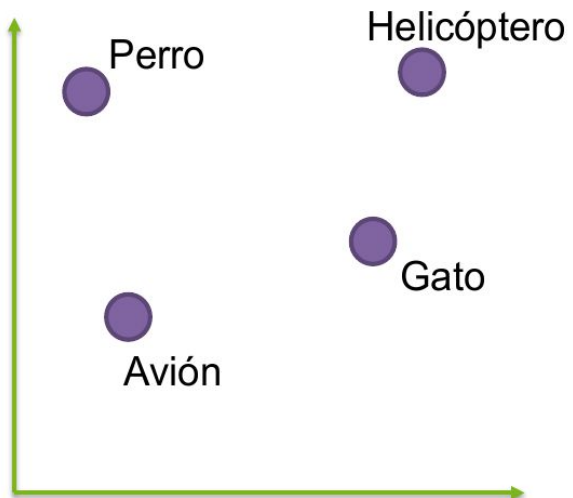
Y esta?

	Variable1		Abeja	Alce	Ardilla	...
1	Abeja		1	0	0	0
2	Alce		0	1	0	0
3	Ardilla		0	0	1	0
4	Águila	→	0	0	0	0
5	Avispa		0	0	0	
6	Avestruz		0	0	0	0
	...		...	...	...	...
949	Zarigüeya		0	0	0	0
950	Zorro		0	0	0	0



- La técnica de embedding consiste en proyectar una entrada en un espacio diferente.
- En redes neuronales, los embeddings son representaciones de variables discretas en un espacio de dimensión baja





¿Qué representación vectorial es mejor?

ID	Tipo	Precio
1	coche	1000
2	libro	1500
3	ordenador	1200
4	camión	2000
5	ordenador	1800
6	coche	2500

One hot encoding:

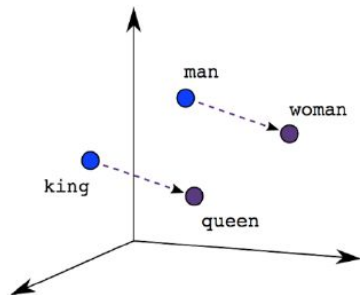
ID	Tipo_coche	Tipo_camión	Tipo_libro	Tipo_ordenador	Precio
1	1	0	0	0	1000
2	0	0	1	0	1500
3	0	0	0	1	1200
4	0	1	0	0	2000
5	0	0	0	1	1800
6	1	0	0	0	2500

ID	Tipo	Precio
1	coche	1000
2	libro	1500
3	ordenador	1200
4	camión	2000
5	ordenador	1800
6	coche	2500

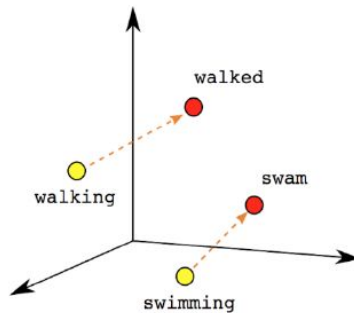
Embeddings:

ID	Tipo	Precio	Embedding_1	Embedding_2	Embedding_3
1	coche	1000	0.2	0.1	-0.1
2	libro	1500	-0.2	0.3	0.1
3	ordenador	1200	0.0	0.1	-0.1
4	camión	2000	0.3	0.2	-0.2
5	ordenador	1800	0.0	0.1	-0.1
6	coche	2500	0.2	0.1	-0.1

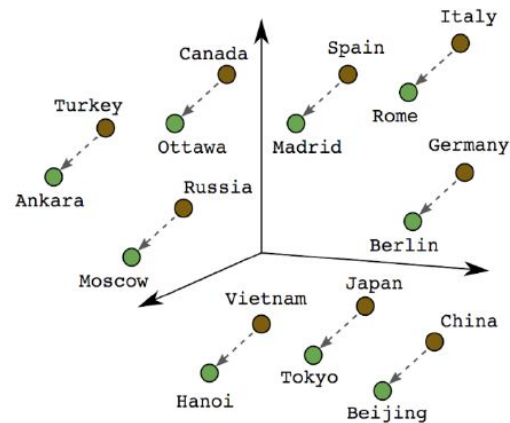
más próximos



Male-Female



Verb Tense



Country-Capital

Rome = [0.91, 0.83, 0.17, ..., 0.41]

Paris = [0.92, 0.82, 0.17, ..., 0.98]

Italy = [0.32, 0.77, 0.67, ..., 0.42]

France = [0.33, 0.78, 0.66, ..., 0.97]

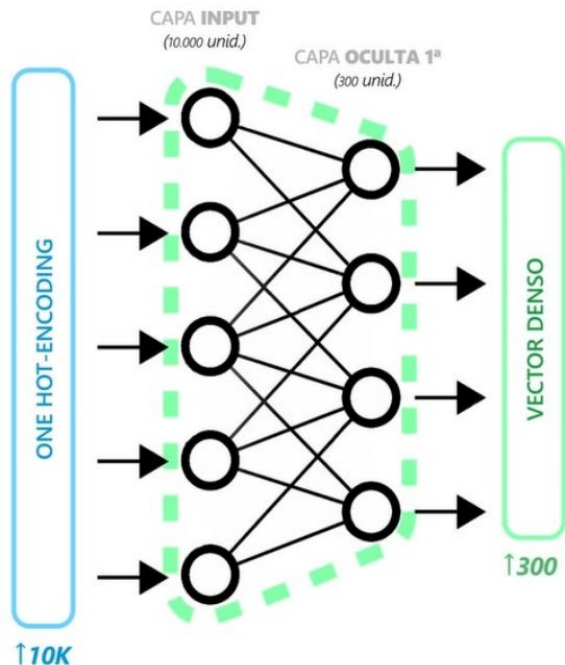
TOKENIZAR

```
sentences = [  
    'Today is a sunny day',  
    'Today is a rainy day',  
    'Is it sunny today?'  
]  
  
{ 'today': 1, 'is': 2, 'a': 3, 'sunny': 4, 'day': 5, 'rainy': 6, 'it': 7 }
```

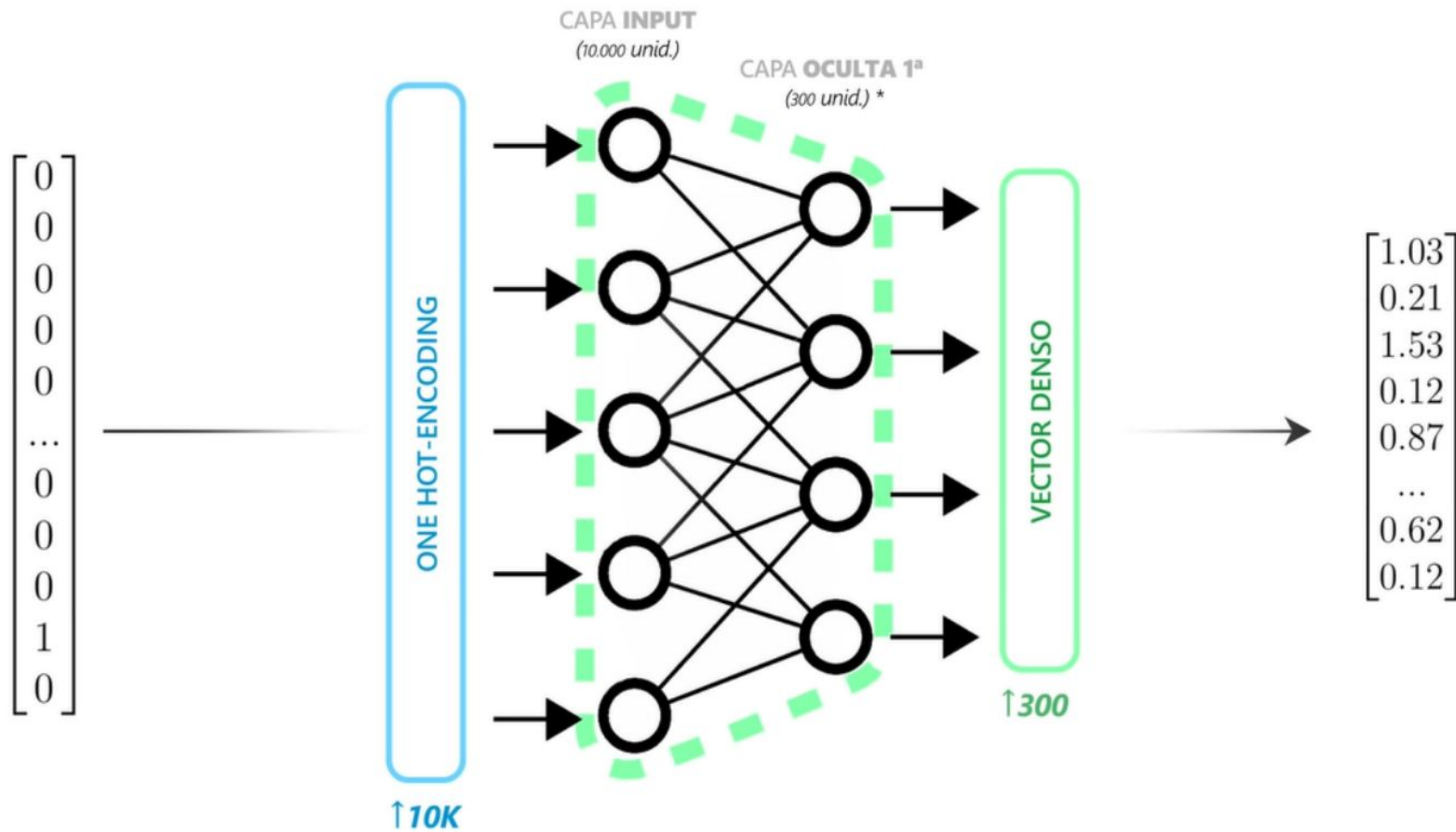


```
[ [1, 2, 3, 4, 5], [1, 2, 3, 6, 5], [2, 7, 4, 1] ]
```

Imaginemos que tenemos un vocabulario con 10.000 palabras:



- Al pasar las palabras por una red neuronal, podemos añadir capas con menos neuronas para comprimir la información
- Tenemos que entrenar la red, dependiendo del tipo de tarea que queramos resolver (análisis de sentimiento, reviews positivas/negativas, etc.)



PADDING

```
sentences = [  
    'Today is a sunny day',  
    'Today is a rainy day',  
    'Is it sunny today?',  
    'I really enjoyed walking in the snow today'  
]
```



```
[  
    [2, 3, 4, 5, 6],  
    [2, 3, 4, 7, 6],  
    [3, 8, 5, 2],  
    [9, 10, 11, 12, 13, 14, 15, 2]  
]
```



```
[ [ 0 0 0 2 3 4 5 6 ]  
  [ 0 0 0 2 3 4 7 6 ]  
  [ 0 0 0 0 3 8 5 2 ]  
  [ 9 10 11 12 13 14 15 2 ] ]
```

Para entrenar un algoritmo de ML siempre tendremos que establecer unos inputs con el mismo número de dimensiones.

Esto se soluciona rellenando con 0s mediante la técnica “padding”

- Ya existen embeddings pre-entrenados que convierte texto a vectores comprimidos
- El más conocido es Word2vec de Google
- <https://projector.tensorflow.org/>

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

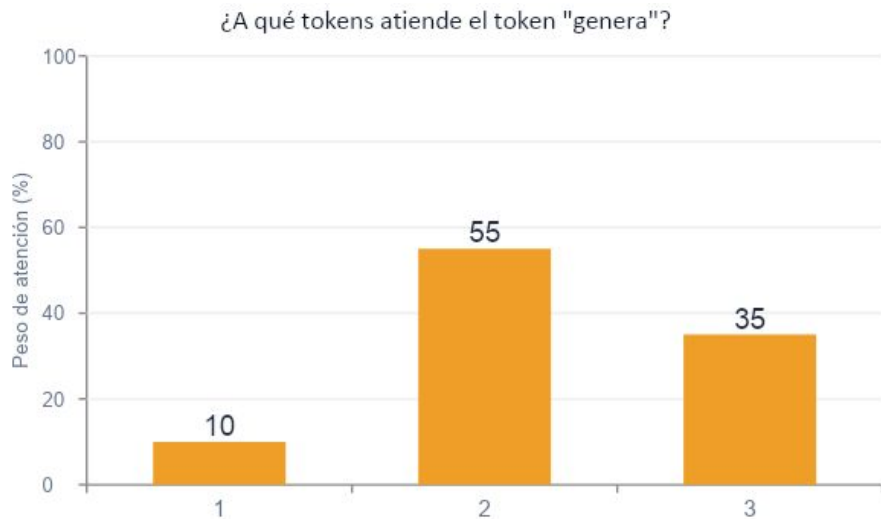
Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* †
illia.polosukhin@gmail.com

Transformers

Lo que define el costo:
attention, KV cache,
tokens

Analogía: Cuando lees "el asegurado debe presentar..." tu cerebro busca el nombre que vio 3 páginas atrás. Attention hace lo mismo: cada token pregunta a los demás "¿cuánto me sirves para entenderme?"



Los pesos suman 100% · causal mask: solo puede atender a tokens anteriores a él

Tres vectores por token:

Q — ¿qué busco?

Busca en los demás tokens la información que necesita para entenderse en contexto.

K — ¿qué ofrezco?

Anuncia qué tipo de información contiene este token a quien lo esté buscando.

V — ¿qué comparto?

El contenido real que transfiere cuando el modelo decide que es relevante para otro token.

Nota: cuanto más largo el prompt, más relaciones calcula el modelo → más costo y tiempo de procesamiento. No incluyas texto que no aporte información relevante.

FASE 1: PREFILL

todos los tokens del prompt (en paralelo)



Q K V

Q K V

Q K V

Q K V

$$Q \cdot K^T / \sqrt{d_k} \rightarrow O(n^2) \text{ en paralelo}$$

Q no se guarda, solo K y V van al cache

K

V

KV CACHE

prefill

K [El]

V [El]

K [modelo]

V [modelo]

K [genera]

V [genera]

K [tokens]

V [tokens]

FASE 2: DECODE

un nuevo token por paso (secuencial)

por

Q K V

+K

+V

decode

K [por]

V [por]

No recalcular lo que ya se procesó

Analogía: Como un bloc de notas donde el modelo va anotando lo que ya leyó. Al generar el token 500, no releer el prompt desde cero — consulta el bloc. Sin cache, generar respuestas largas sería prohibitivo.

Sin KV cache

Para cada token nuevo, recalcular K y V de todos los tokens anteriores. Con 1,000 tokens previos: 1,000 recálculos por cada token generado.

Con KV cache

K y V de cada token se guardan la primera vez. Los tokens siguientes solo calculan sus propios K/V y reutilizan el cache.

Prompt caching

Si el system prompt no cambia, el provider lo cachea entre requests. Hasta 90% de descuento en esos tokens. Anthropic y OpenAI lo soportan.

40 GB

KV cache — Llama 70B @ 128K tokens, fp16

Regla de diseño de prompts:

Contenido estático (system prompt, instrucciones, few-shots) → **primero**.

Contenido dinámico

(pregunta del usuario, documentos que cambian) → **al final**.

Analogía: Los modelos aprendieron con más texto en inglés → tienen más "piezas" en inglés → el español se parte en más fragmentos → más tokens por la misma información → más costo.

+30%

más tokens en español
vs el mismo contenido en inglés

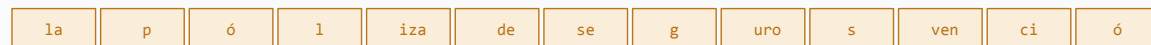
Ejemplo con cl100k (GPT-4) — misma frase:

EN → "insurance policy expired"



= 4 tokens

ES → "la póliza de seguros venció"



= 10 tokens (+4 = +67%)

Nota:

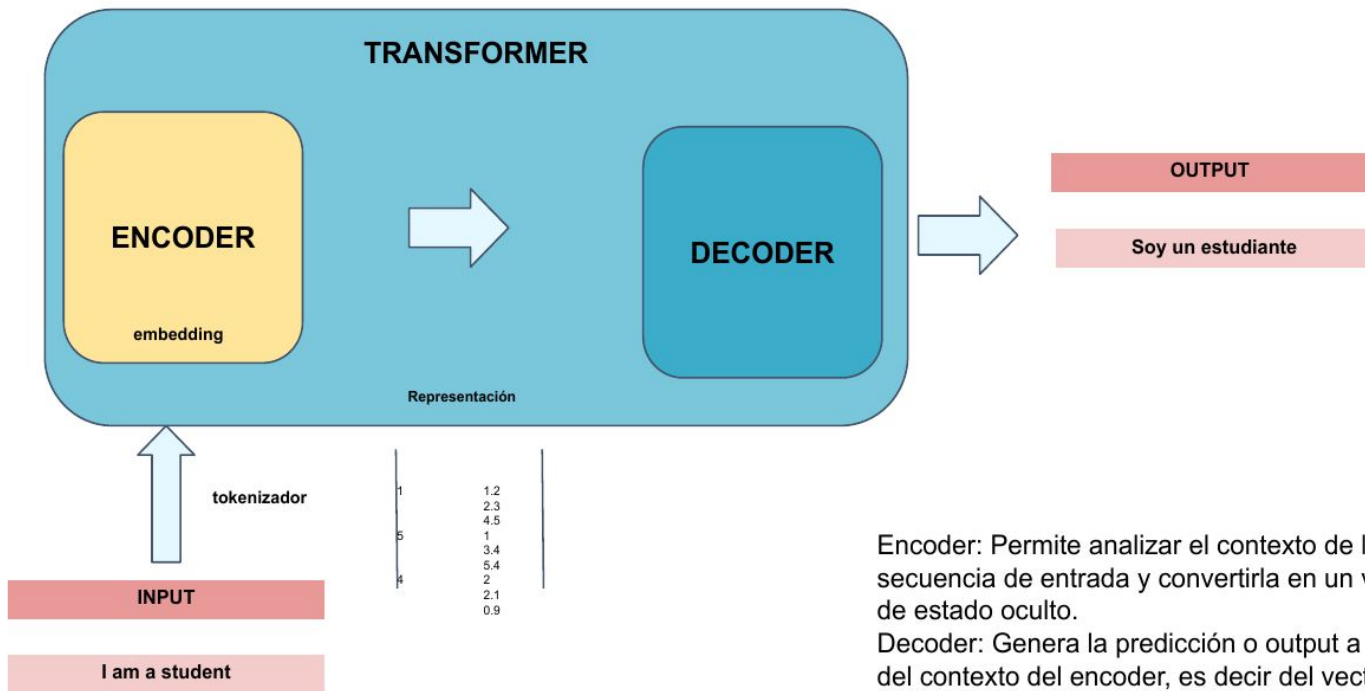
Nunca estimes costo multiplicando caracteres × constante. Tokeniza tu corpus real con el tokenizer del modelo que vayas a usar.

Tokens para el mismo reporte de siniestro (por tokenizer)



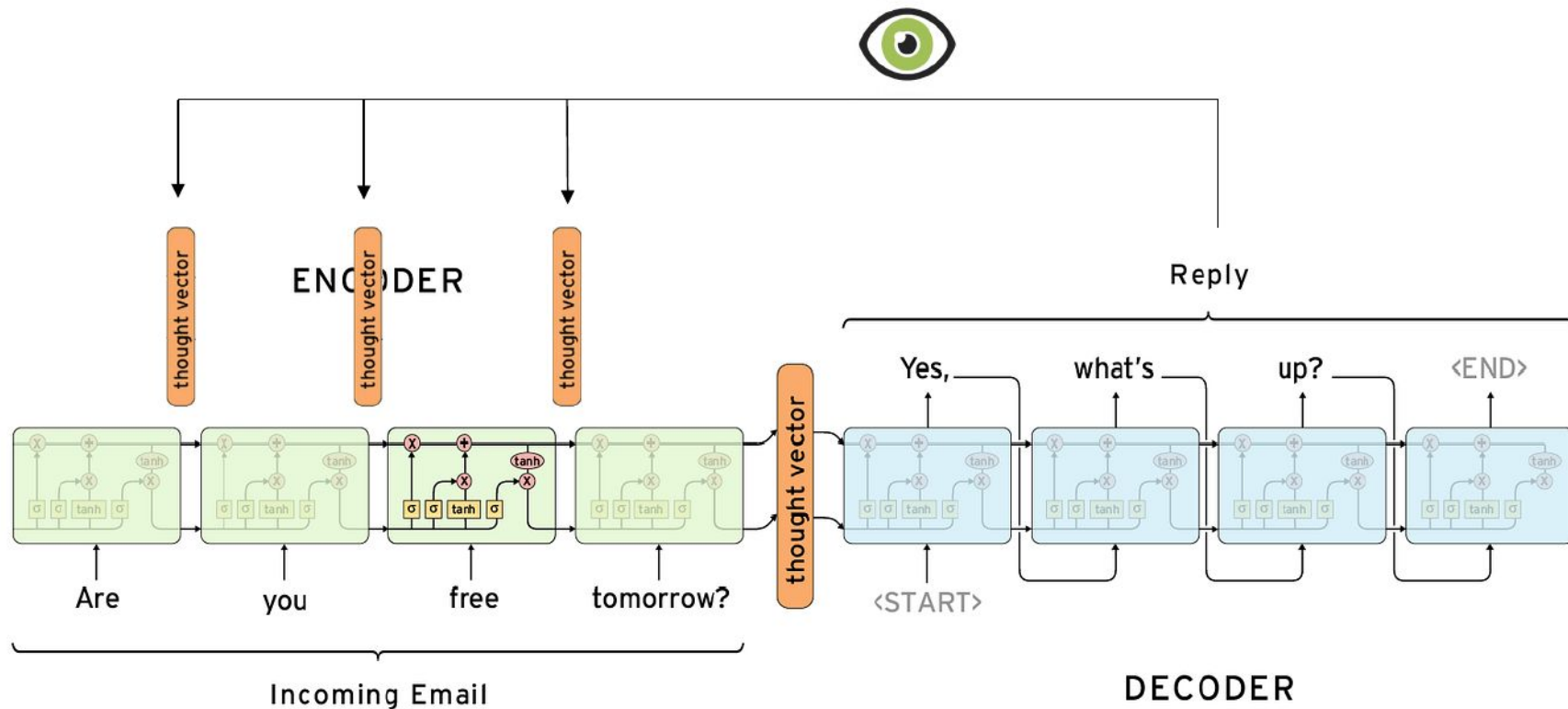
Transformers

Los transformers consisten en un encoder y un decoder. El input pasa por el encoder y genera una representación matricial de este. El decoder coge esa representación y genera un output.



Encoder: Permite analizar el contexto de la secuencia de entrada y convertirla en un vector de estado oculto.

Decoder: Genera la predicción o output a partir del contexto del encoder, es decir del vector de estado oculto del encoder.



Los mecanismos de atención ofrecen una interfaz entre el encoder y el decoder para acceder a los estados ocultos de cada paso

Machine learning

- Supervisados
- No Supervisados
- Auto supervisados

La auto-supervisión es un método de entrenamiento en Machine Learning (ML) donde el propio modelo puede inferir las etiquetas (datos para aprender) directamente a partir de los datos de entrada, en lugar de requerir etiquetas explícitas creadas por humanos

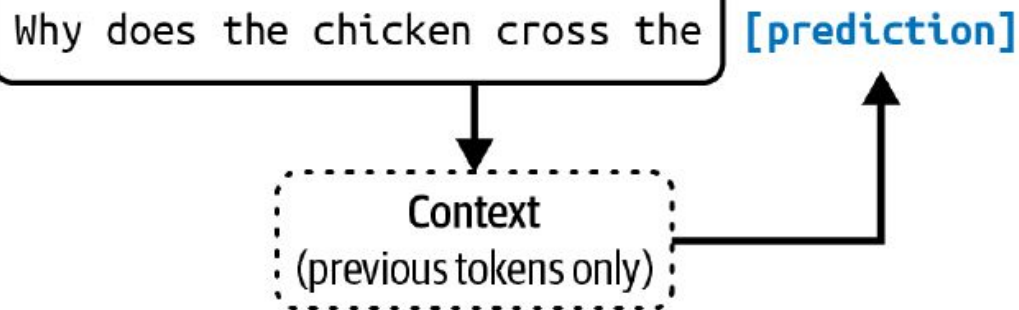
Masked language:

- Se entrena para predecir tokens (palabras o partes de palabras) que faltan en cualquier lugar de una secuencia.
- Utiliza el contexto tanto anterior como posterior a los tokens perdidos para hacer su predicción, actuando como un "rellenador de espacios en blanco."
- Se usa para tareas no generativas, como el análisis de sentimientos, la clasificación de texto y la depuración de código, donde se requiere una comprensión del contexto general.
- Ejemplo: BERT

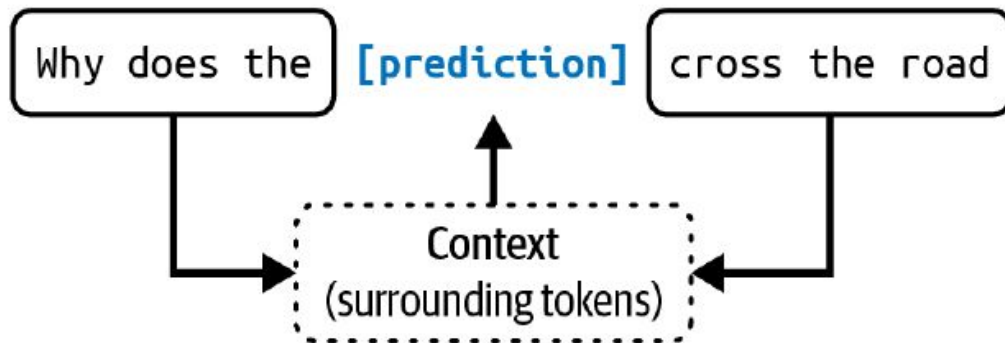
Autoregressive Language Model:

- Se entrena para predecir el siguiente token en una secuencia, basándose únicamente en los tokens que lo preceden.
- Puede generar un token tras otro de forma continua
- Son los modelos preferidos para la generación de texto y, por esta razón, son mucho más populares que los modelos de lenguaje enmascarado.
- Ejemplo: GPT-4, Claude, LLaMA

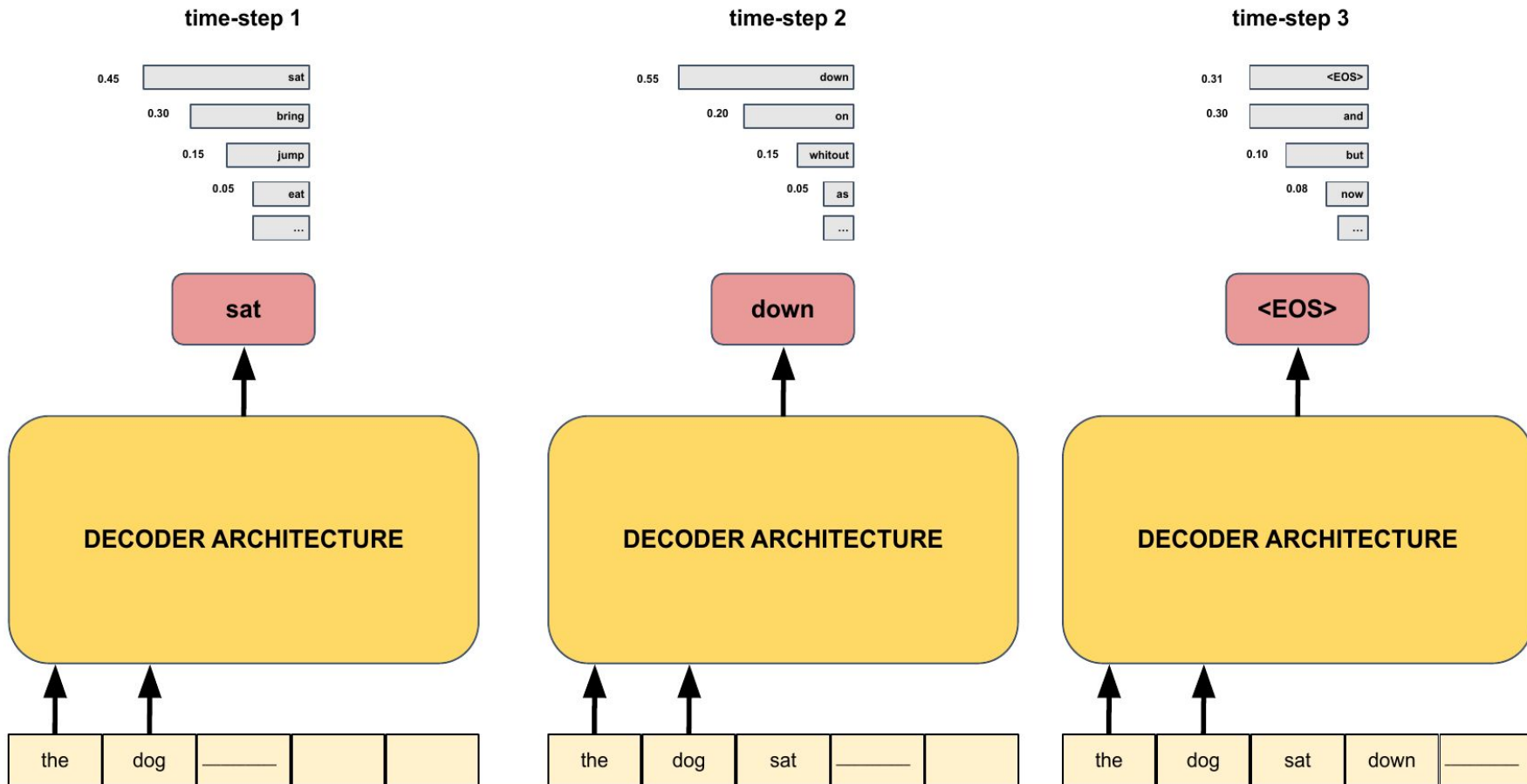
Autoregressive LM



Masked LM



Fuente: "AI Engineering", Chip Huyen (2025)



GPT-2

Language Models are Unsupervised Multitask Learners

Alec Radford^{*1} Jeffrey Wu^{*1} Rewon Child¹ David Luan¹ Dario Amodei^{**1} Ilya Sutskever^{**1}

Abstract

Natural language processing tasks, such as question answering, machine translation, reading comprehension, and summarization, are typically approached with supervised learning on task-specific datasets. We demonstrate that language models begin to learn these tasks without any explicit supervision when trained on a new dataset of millions of webpages called WebText. When conditioned on a document plus questions, the answers generated by the language model reach 55 F1 on the CoQA dataset - matching or exceeding the performance of 3 out of 4 baseline systems without using the 127,000+ training examples. The capacity of the language model is essential to the success of zero-shot task transfer and increasing it improves performance in a log-linear fashion across tasks. Our largest model, GPT-2, is a 1.5B parameter Transformer that achieves

competent generalists. We would like to move towards more general systems which can perform many tasks – eventually without the need to manually create and label a training dataset for each one.

The dominant approach to creating ML systems is to collect a dataset of training examples demonstrating correct behavior for a desired task, train a system to imitate these behaviors, and then test its performance on independent and identically distributed (IID) held-out examples. This has served well to make progress on narrow experts. But the often erratic behavior of captioning models (Lake et al., 2017), reading comprehension systems (Jia & Liang, 2017), and image classifiers (Alcorn et al., 2018) on the diversity and variety of possible inputs highlights some of the shortcomings of this approach.

Our suspicion is that the prevalence of single task training on single domain datasets is a major contributor to the lack of generalization observed in current systems. Progress

text generation

GPT-3

Language Models are Few-Shot Learners

Tom B. Brown [*]	Benjamin Mann [*]	Nick Ryder [*]	Melanie Subbiah [*]	
Jared Kaplan [†]	Prafulla Dhariwal	Arvind Neelakantan	Pranav Shyam	Grish Sastry
Amanda Askell	Sandhini Agarwal	Ariel Herbert-Voss	Gretchen Krueger	Tom Henighan
Rewon Child	Aditya Ramesh	Daniel M. Ziegler	Jeffrey Wu	Clemens Winter
Christopher Hesse	Mark Chen	Eric Sigler	Mateusz Litwin	Scott Gray
Benjamin Chess	Jack Clark	Christopher Berner		
Sam McCandlish	Alec Radford	Ilya Sutskever	Dario Amodei	

(vs)

OpenAI

Abstract

Recent work has demonstrated substantial gains on many NLP tasks and benchmarks by pre-training on a large corpus of text followed by fine-tuning on a specific task. While typically task-agnostic in architecture, this method still requires task-specific fine-tuning datasets of thousands or tens of thousands of examples. By contrast, humans can generally perform a new language task from only a few examples or from simple instructions – something which current NLP systems still largely struggle to do. Here we show that scaling up language models greatly improves task-agnostic, few-shot performance, sometimes even reaching competitiveness with prior state-of-the-art fine-tuning approaches. Specifically, we train GPT-3, an autoregressive language model with 175 billion parameters, 10x more than any previous non-sparse language model, and test its performance in the few-shot setting. For all tasks, GPT-3 is applied without any gradient updates or fine-tuning, with tasks and few-shot demonstrations specified purely via text interaction with the model. GPT-3

Keep conversations



Nace GPT y lo cambia todo



datapath

GPT-3

Language Models are Few-Shot Learners

Tom B. Brown*	Benjamin Mann*	Nick Ryder*	Melanie Subbiah*	
Jared Kaplan†	Prafulla Dhariwal	Arvind Neelakantan	Pranav Shyam	Girish Sastry
Amanda Askell	Sandhini Agarwal	Ariel Herbert-Voss	Gretchen Krueger	Tom Henighan
Rewon Child	Aditya Ramesh	Daniel M. Ziegler	Jeffrey Wu	Clemens Winter
Christopher Hesse	Mark Chen	Eric Sigler	Mateusz Litwin	Scott Gray
Benjamin Chess		Jack Clark	Christopher Berner	
Sam McCandlish	Alec Radford	Ilya Sutskever	Dario Amodei	

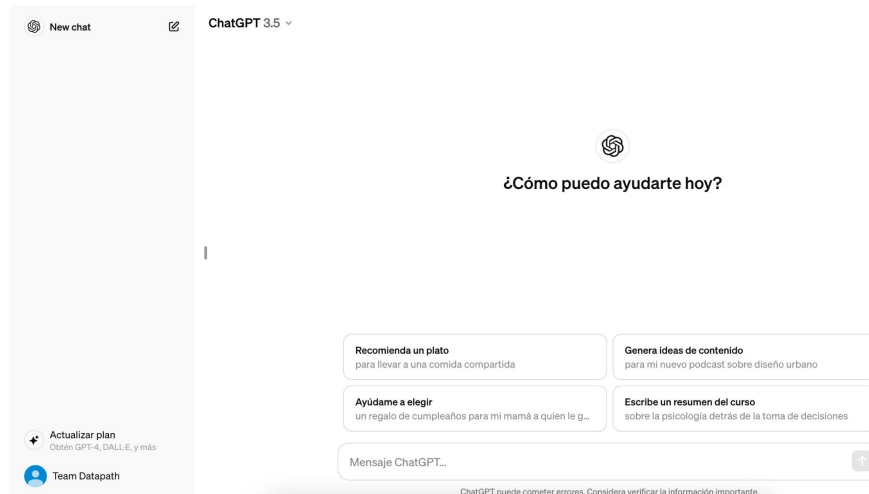
OpenAI

Abstract

Recent work has demonstrated substantial gains on many NLP tasks and benchmarks by pre-training on a large corpus of text followed by fine-tuning on a specific task. While typically task-agnostic in architecture, this method still requires task-specific fine-tuning datasets of thousands or tens of thousands of examples. By contrast, humans can generally perform a new language task from only a few examples or from simple instructions – something which current NLP systems still largely struggle to do. Here we show that scaling up language models greatly improves task-agnostic, few-shot performance, sometimes even reaching competitiveness with prior state-of-the-art fine-tuning approaches. Specifically, we train GPT-3, an autoregressive language model with 175 billion parameters, 10x more than any previous non-sparse language model, and test its performance in the few-shot setting. For all tasks, GPT-3 is applied without any gradient updates or fine-tuning, with tasks and few-shot demonstrations specified purely by text interaction with the model. GPT-3



Interactive interface



ChatGPT

OpenAI

User Mode

ChatGPT



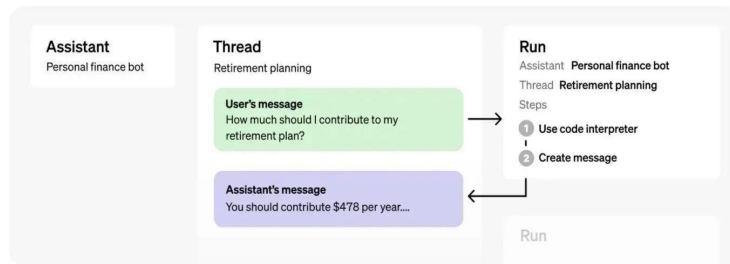
Interact with our flagship language models in a conversational interface



Developer Mode

User Level (Software as a Service)

Coding Level



OpenAI Assistants

API



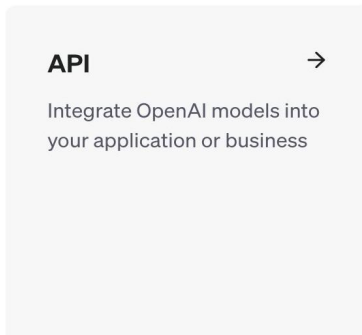
Integrate OpenAI models into your application or business

Developer Mode

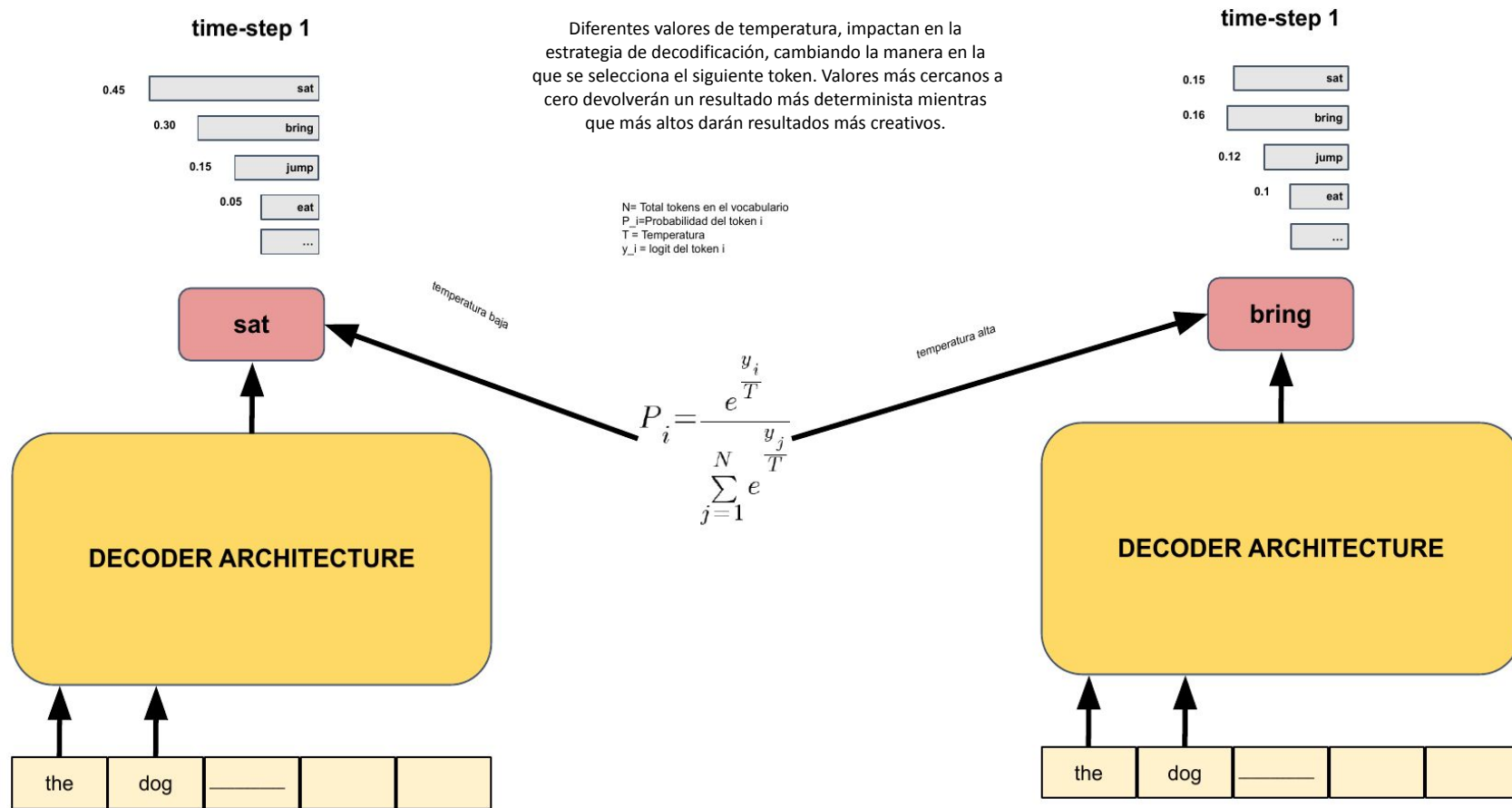
User Level (Software as a Service)



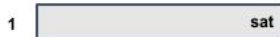
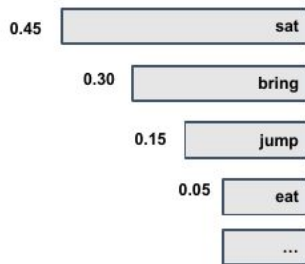
Coding Level



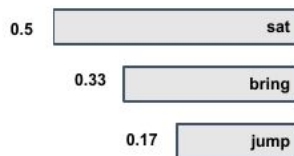
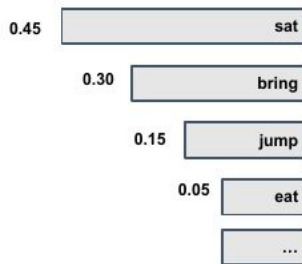
“Only pay for what you use”



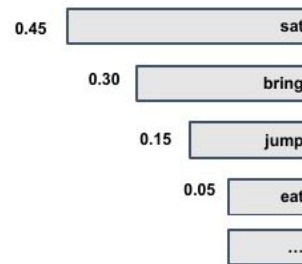
top-k(1)
greedy decoding



top-k(3)



top-p(0.8)





<https://docs.docker.com/desktop/setup/install/windows-install/>



Visual Studio Code

<https://code.visualstudio.com/download>



Antigravity

<https://antigravity.google/download>



python



ANACONDA®

<https://ernestos-private-organization.gitbook.io/ai-engineer/instalacion-de-anaconda>



¡Gracias!

Aprende, aplica y crece